Contents

Multi-Tenancy — Isolation Models, Enforcement, and Failure Modes

Last updated: 2026-08-11

What this note is for: a revision reference for interviews and for designing a new system from scratch. Concepts first, then the eng-labs implementation as a worked case study (§8).

Companion notes: eng-labs-platform — what the system is. multitenant-platform-engineering-lessons — what went wrong and the fix roadmap.

---

0. The framing that makes everything else simple

Multi-tenancy = many customers (tenants) share one system. Every design question reduces to two:

> **Where does tenant data live?** → the *isolation model* **What stops a query crossing the boundary?** → the *enforcement layer*

These are **two independent axes**, and conflating them is the most common mistake. You pick one from each:

	Options
Isolation model (§1)	silo · database-per-tenant · schema-per-tenant · pooled · hybrid
Enforcement layer (§3)	application code · ORM/repository · database RLS · separate connection/instance

A pooled model with RLS enforcement is far safer than a pooled model with hand-written filters — same data layout, completely different risk profile. Say both when you answer.

Tenant vs user vs organisation. A *tenant* is the unit of isolation and usually the unit of billing — the customer. Users belong to a tenant. Anything that must never be visible across the boundary defines what the tenant is.

---

1. The five isolation models

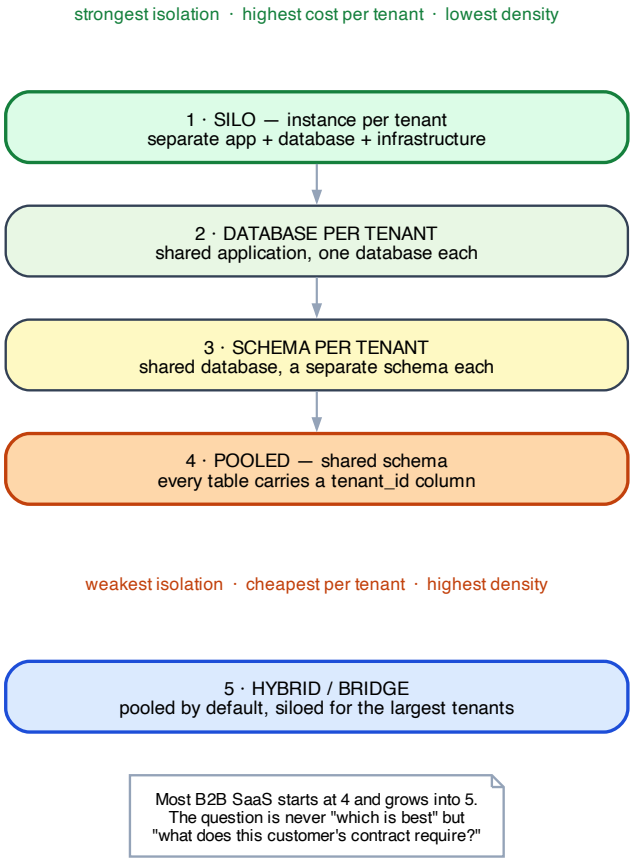

Figure 1: The five multi-tenancy isolation models

The comparison table (the thing to memorise)

	1 Silo	2 DB per tenant	3 Schema per tenant	4 Pooled	5 Hybrid
Isolation strength	strongest	strong	medium	weakest	per-tier
Cost per tenant	highest	high	medium	lowest	mixed
Density (tenants/host)	1	tens	hundreds	unlimited	mixed
Noisy-neighbour risk	none	low	medium	high	per-tier

	1 Silo	2 DB per tenant	3 Schema per tenant	4 Pooled	5 Hybrid
Per-tenant schema variation	easy	easy	easy	hard	tiered
Blast radius of a code bug	1 tenant	1 tenant	1 tenant	**all tenants**	tiered
Compliance / data residency	trivial	easy	awkward	hard	easy for the tier that needs it
Migration effort	N deploys	N databases	N schemas	**1**	1 + N
Onboarding a tenant	slow (minutes–hours)	medium	fast	**instant**	mixed
Per-tenant delete / export (GDPR)	trivial	trivial	easy	**hard**	mixed
Cross-tenant analytics	very hard	hard	medium	**trivial**	medium
Ops complexity	highest	high	medium	lowest	highest

Notice the table is almost perfectly **inverted** between isolation and efficiency. That's the whole subject: there is no free lunch, only a choice about which pain you accept.

1 · Silo — full instance per tenant

Separate app deployment, separate database, often separate cloud account/VPC.

- **Use when:** enterprise contracts demand it, data must stay in a specific region, or a regulator requires physical separation. Also the honest answer for a handful of very large customers.
- **Cost:** you now operate N systems. Every release is N deploys; a hotfix is N rollouts; version skew across tenants becomes real.
- **Watch:** this is usually where "we'll just spin up another instance" quietly becomes an ops team.

2 · Database per tenant — shared application

One application fleet; the app resolves the tenant to its own database connection.

- **Use when:** you need strong data isolation and easy per-tenant backup/restore/export, but want one codebase.
- **Cost:** connection-pool pressure (N pools), and migrations must run N times — with partial-failure handling. Restoring one tenant is trivial, which is a genuinely underrated benefit.
- **Watch:** connection limits are the practical ceiling. Hundreds of tenants is fine; tens of thousands is not.

3 · Schema per tenant — shared database

One database, one Postgres schema (namespace) per tenant. The app sets `search_path` per request.

- **Use when:** a middle ground — better isolation than pooled, cheaper than a database each.
- **Cost:** the awkward one in practice. Migrations still run N times, catalog bloat becomes a real problem past a few thousand schemas, and many tools and ORMs handle it badly.
- **Watch:** often chosen as a compromise and regretted. Be able to say *why* you'd skip it.

4 · Pooled — shared schema, `tenant_id` discriminator

Every tenant-owned table carries a `tenant_id` column. Every query filters on it. **This is what most B2B SaaS runs, including eng-labs.**

- **Use when:** many tenants, fast self-serve onboarding, one migration path, and cross-tenant analytics matter.
- **Cost:** isolation is now a *property of your code*, not of your infrastructure. One missing `WHERE` clause is a data breach. Noisy neighbours share indexes and connections. Per-tenant deletion means finding every table.
- **Non-negotiable mitigations:** enforce at the ORM or database layer (§3), never at the call site; index `(tenant_id, …)` as a *leading* column on every hot query; test the cross-tenant negative case (§7).

5 · Hybrid / bridge — pooled by default, siloed on demand

Small and mid customers pooled; enterprise customers get their own database or instance. Same codebase, tenant metadata records which tier a tenant is in.

- **Use when:** you've succeeded. This is the natural end state of a growing SaaS — one enterprise contract demands isolation and you don't want to re-architect for everyone.
- **Cost:** you now maintain **both** paths, including two migration strategies and two backup stories. Highest ops complexity of the five.
- **Design implication:** if you think you'll ever get here, **make tenant→datasource resolution an abstraction from day one**, even when it always returns the same connection. Retrofitting that indirection later is the expensive part.

---

2. How to choose — the decision guide

Ask these in order. The first "yes" that carries a contractual or legal obligation wins.

1. **Does any contract or regulation require physical separation or data residency?** → silo or DB-per-tenant for those tenants (which means hybrid).
2. **How many tenants, realistically, in 3 years?** Tens → DB-per-tenant is viable. Thousands+ → pooled.
3. **How is it sold?** Self-serve signup demands instant onboarding → pooled. Enterprise sales with onboarding calls → per-tenant is affordable.
4. **Do tenants need structurally different data models?** Genuinely different → per-tenant schema. Same shape, different labels → pooled with configuration. (See the lessons note §8.2 — flexibility must earn its complexity.)
5. **Is cross-tenant analytics a product feature?** → pooled makes it trivial; everything else makes it a data-warehouse project.

6. **What's the blast radius you can survive?** Pooled means one bad query can affect everyone.

The honest default: start pooled, enforce at the database or ORM layer, and keep tenant→datasource resolution behind an interface so hybrid stays available.

---

3. Where isolation is enforced — the axis that actually decides safety

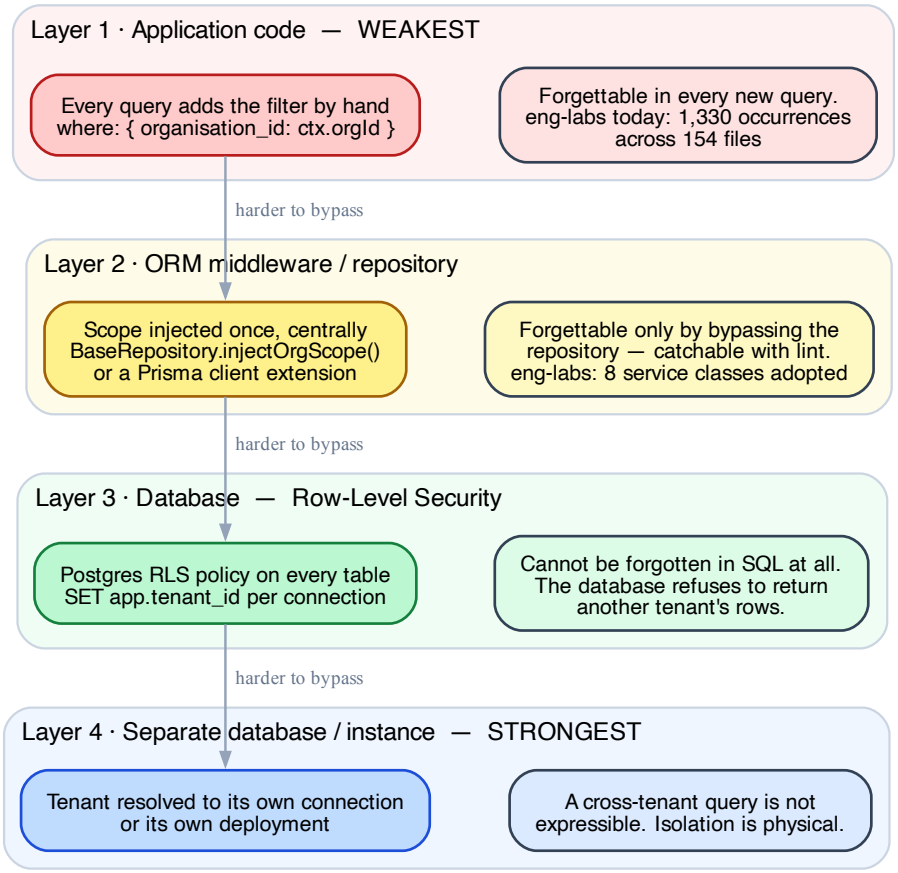

Figure 2: Where tenant isolation can be enforced

Layer	Mechanism	Can a developer forget it?
1 · Application code	every query hand-writes `where: { tenant_id }`	**Yes — in every new query.** No safety net.
2 · ORM / repository	one central scope injector (Prisma client extension, base repository, Django manager, Rails `default_scope`)	Only by bypassing the layer — and lint can catch that.

Layer	Mechanism	Can a developer forget it?
3 · Database (RLS)	Postgres Row-Level Security policy per table; `SET app.tenant_id` per connection	**No.** The database refuses the rows. Even raw SQL is covered.
4 · Separate connection / instance	tenant resolved to its own datasource	**No.** A cross-tenant query isn't expressible.

Row-Level Security, concretely — worth being able to sketch:

```
ALTER TABLE courses ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON courses
  USING (tenant_id = current_setting('app.tenant_id')::text);

-- then per request / per connection:
SET LOCAL app.tenant_id = 'tnt_abc123';
```

Every subsequent query on that connection is filtered by the database itself. The trade-offs to mention: it needs the setting applied reliably on every connection (easy to get wrong with pooling), it can complicate query plans, and you need a deliberate bypass role for migrations and admin tooling.

The principle underneath all of this: *if a rule must be obeyed everywhere, make it impossible to disobey rather than documenting it.* Layer 1 is documentation. Layers 2–4 are enforcement.

---

4. Where leaks actually happen

Real cross-tenant incidents almost never come from the main CRUD path — that's the code everyone reviews. They come from these six:

Leak path	Why it happens	Fix
Background jobs / queues	there's no request, so no `req.context` to read	serialise `tenant_id` into the job payload; make the worker's data access require it
Reports & aggregates	a `COUNT/SUM/GROUP BY` written directly, filter forgotten	route analytics through the same scoped layer; never hand-write aggregate SQL
Cache keys	key built from entity id without the tenant	make `tenant_id` a mandatory part of every cache key by construction
Admin / impersonation paths	deliberately cross-tenant, then copy-pasted into normal code	keep them in a separate, clearly-named module; audit-log every use
`findUnique` by id alone	ids are globally unique (e.g. nanoid), so the filter feels unnecessary — until an id leaks or is guessed	always scope by tenant *as well as* id; treat id as insufficient authorisation
Raw SQL & migrations	bypass the ORM, so layer-2 enforcement doesn't apply	this is the strongest argument for database-level RLS

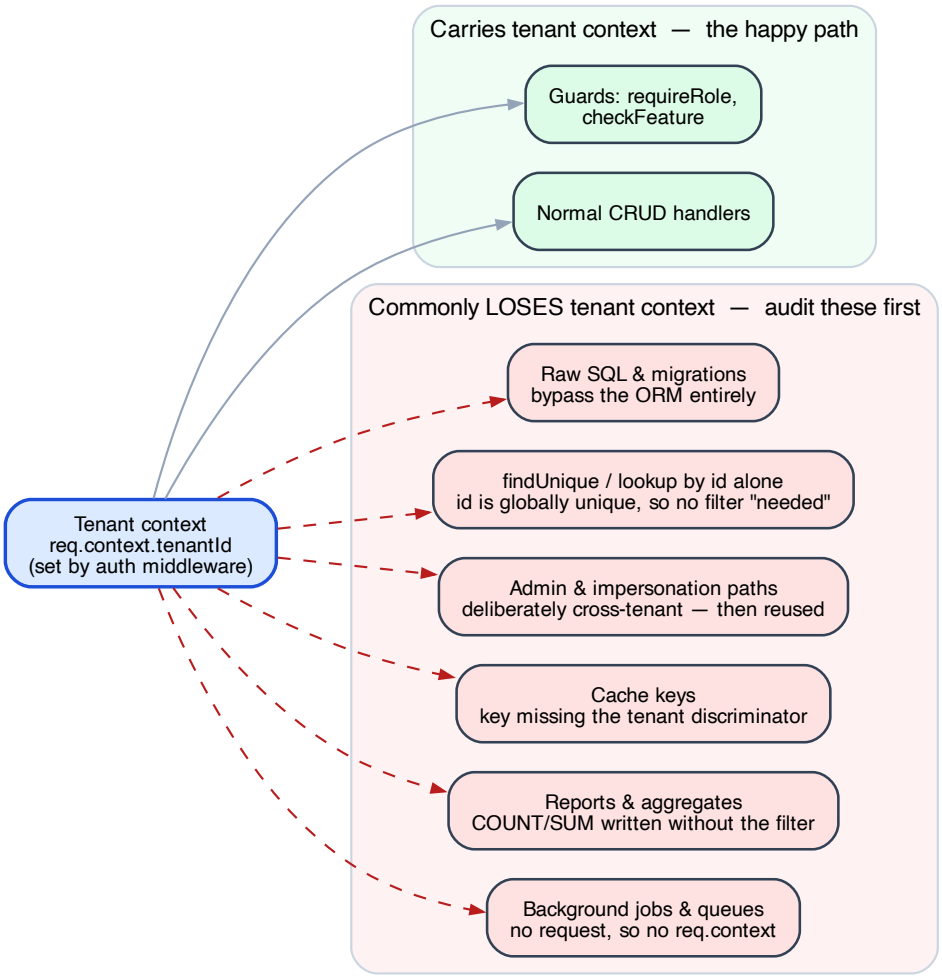

Figure 3: Paths that lose tenant context

The `findUnique` one is the subtle killer, and eng-labs hit exactly this: `BaseRepository.findUnique()` is deliberately downgraded to a scoped `findFirst()` because `findUnique` cannot enforce `organisation_id` without changing the unique constraints. Good instinct, and a good interview anecdote.

---

5. "Tenant" touches more than the database

A checklist people forget — the data layer is maybe half the work:

Concern	What it needs
Tenant resolution	how a request declares its tenant: subdomain, path prefix, custom domain, JWT claim, or header. Pick one canonical source; treat the others as untrusted.
Auth & context	tenant on the token/session; validate the user is actually mapped to the tenant they claim (don't trust a header)
Feature gating	which products/modules each tenant has; usually a registry table + a per-tenant enablement table
Config & branding	logo, domain, email templates, locale, timezone
Quotas & rate limits	per-tenant limits, so one tenant can't exhaust shared capacity (the noisy-neighbour mitigation)
Billing & metering	usage counted per tenant, reconciled with the plan
Observability	`tenant_id` on every log line, trace span and metric — otherwise you cannot debug "it's slow for customer X"
File storage	tenant-prefixed paths/buckets; signed URLs scoped to the tenant
Search indexes	tenant as a mandatory filter, or an index per tenant
Data lifecycle	per-tenant export and hard-delete (GDPR); hardest in the pooled model, so design it early

---

6. Operations, per model — what interviews probe

"How do you run a migration across 500 tenant databases?" The expected answer: a migration runner that iterates tenants with per-tenant state tracking, runs in batches, is idempotent and resumable, and tolerates partial failure (tenant 213 failed; the other 499 succeeded and you can retry just that one). Plus a compatibility rule: deploy schema changes that are backward-compatible so app and schema versions can differ mid-rollout.

"How do you restore one tenant's data?" Trivial in models 1–2 (restore that database). In pooled, you need either per-tenant logical exports or point-in-time recovery into a scratch instance followed by a filtered copy. Worth knowing this is a real weakness of pooled.

"How do you onboard a tenant?" Pooled: insert a row. Per-database: provision, migrate, seed, register — a workflow with failure states.

Backward-compatible schema change order (applies to any model, asked constantly): add nullable column → deploy code that writes both → backfill → deploy code that reads new → make non-null / drop old. Never one big-bang migration.

---

7. Testing tenant isolation

The single highest-value test category in a multi-tenant system, and cheap to write:

```
For every endpoint that reads or writes tenant data:
  1. 200  - tenant A's user reads tenant A's resource            (happy path)
  2. 401  - no token                                              (authentication)
  3. 403  - tenant A's user, insufficient role                    (authorisation)
  4. 404/403 - tenant A's user requests tenant B's resource by id (ISOLATION)
```

Cases 1–3 are "triangle coverage." **Case 4 is the one that catches breaches**, and it's the one usually missing. A pooled system without case-4 tests has no evidence its isolation works.

Also worth having: a repository-level test that a query built without tenant context **throws** rather than returning everything. Fail closed, loudly.

---

8. Case study — eng-labs (model 4, pooled)

The hierarchy is four levels, not two

```
Organisation  →  Entity (tenant)  →  Unit  →  Users
```

Most multi-tenancy writing assumes `tenant → user`. Real B2B often needs more: here an Organisation may own several Entities (colleges), each with Units (departments, sections, batches). **Consequence: "scoped" is not one column but a path.** Queries scope by `organisation_id` and typically `tenant_id`, and sometimes need `unit_id` too — which is exactly why the leak surface is large (154 files).

Lesson for a new system: decide how many levels the boundary has *before* the schema, and give each level a first-class model. Retrofitting a level (see the missing cohort/batch level in the lessons note §2.1) is expensive.

Feature gating: two gates, both must pass

```
feature_registry.enabled     ← platform-level kill switch (we control)
        AND
entity_features.enabled      ← tenant-level entitlement (the customer's contract)
```

A clean design worth reusing: it separates "does this product exist" from "has this customer bought it," and lets support disable a broken feature globally without touching entitlements.

Enforcement: currently split, which is the problem

Layer	Status
1 · application code	**dominant** — 1,330 `organisation_id` occurrences across 154 files
2 · ORM / repository	**partial** — `BaseRepository.injectOrgScope()` exists; 8 service classes adopted
3 · database RLS	not used
4 · separate connection	not applicable (pooled)

The fix is not a rewrite: move `injectOrgScope` into a **Prisma client extension** so it applies without opt-in. Same data model, enforcement moves from layer 1 to layer 2, and the 154-file surface stops mattering.

The naming trap — the most valuable thing in this note

Throughout the codebase, `tenant_id` means `organisation_entities.id`. But `organisation_entities` **also has a column literally called `tenant_id`**, a legacy external identifier that must never be used for FKs or queries.

So the name is correct on every table except the one you'd consult to learn what it means. Nothing in the type system warns you. **A name that is right everywhere except at its own definition is worse than a bad name — it's a trap.** Rename it (`external_tenant_ref`) and an entire class of bug disappears.

What I'd do differently, in order

1. Enforce scoping at layer 2 from day one (client extension), never at call sites.
2. Model the hierarchy's real levels — including cohort/batch and time — before writing the schema.
3. One canonical name per concept; rename the legacy column immediately.
4. Write case-4 isolation tests alongside the first endpoint, not later.
5. Keep tenant→datasource resolution behind an interface, so hybrid stays cheap if an enterprise contract ever demands a silo.

---

9. Compact interview answers

"How would you design multi-tenancy for a new B2B SaaS?" > Two decisions, not one: the isolation model and the enforcement layer. I'd default to pooled — shared schema with a tenant discriminator — because it gives instant onboarding, one migration path, and easy cross-tenant analytics. But pooled makes isolation a property of the code, so I'd enforce it in the data layer, not at call sites: a client extension or Postgres RLS, so a query that forgets the tenant filter is impossible rather than merely discouraged. I'd also keep tenant→datasource resolution behind an interface from day one, so when an enterprise customer demands a dedicated database I can go hybrid without re-architecting.

"What's the biggest risk in the pooled model?" > One missing `WHERE` clause is a data breach, and it won't come from the CRUD path everyone reviews — it comes from background jobs with no request context, hand-written report aggregates, cache keys missing the tenant, or a lookup by a globally-unique id where the filter felt unnecessary. That's why I want enforcement below the application layer, and a test on every endpoint that tenant A gets a 404 for tenant B's resource.

"Tell me about a multi-tenancy mistake you made." > On a platform with 199 models, we enforced tenant scoping in application code — the rule was documented, and every query restated it by hand. It ended up written out in 154 files. Nothing was wrong functionally, but any change to what "scoped" means became a 154-file edit with no compiler help, and each new query was a fresh chance to forget. We'd already written the right abstraction — a base repository that injects the scope once — but it was opt-in, so adoption stalled at 8 service classes. The lesson: if a rule must hold everywhere, make it impossible to break rather than documenting it. The fix is a client extension so it applies without opt-in.

"How do you migrate 500 tenant databases?" > Backward-compatible changes only, so app and schema versions can differ during rollout — add nullable, dual-write, backfill, switch reads, then clean up. And a migration runner that's idempotent, resumable, batched, and tracks state per tenant, so a failure on tenant 213 doesn't block the other 499 or force a full re-run.

---

10. Design checklist for a new multi-tenant system

- ☐ Name the tenant explicitly — what must *never* cross this boundary?
- ☐ How many levels does the hierarchy have? Model each as first-class.
- ☐ Pick the isolation model (§2 decision guide) and write down *why*.
- ☐ Pick the enforcement layer — **not** application code.
- ☐ One canonical name for the tenant key; the same name everywhere, including its own table.
- ☐ Tenant→datasource resolution behind an interface, even if it's a constant today.
- ☐ Index `(tenant_id, …)` as a leading column on every hot query path.
- ☐ Tenant context flows into background jobs, cache keys, logs, traces, metrics, storage paths, search.
- ☐ Case-4 isolation test on the first endpoint, and every one after.
- ☐ Repository throws — loudly — if tenant context is missing. Fail closed.
- ☐ Per-tenant export and hard-delete designed before you need them.
- ☐ Per-tenant quotas/rate limits, so no tenant can exhaust shared capacity.
- ☐ Feature entitlement separated from platform kill-switch.